WebRTC

IF3110 – Web-based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Opening question

How can two browsers exchange live video?



What must they exchange before media can flow?

Opening question

How can two browsers exchange live video?

Traditional Model (HTTP/WS)

All media must go through a server:

- **Server Bottleneck:** server bandwidth increases drastically.
- **High Latency:** video latency increases (500ms – 5 detik).
- **Privacy Risk:** Data stream is stored in the server.

WebRTC Model (Peer-to-Peer)

Browser A & B directly connected:

- **Direct Media Stream:** Audio/video packages stream directly without server.
- **Ultra-Low Latency:** Latency can be below 100–200ms.
- **No Server Cost:** The server does not bear the video load.

Is peer-to-peer audio/video communication always good in all situation?

Core WebRTC capabilities

Media

Audio, video, and screen sharing travel as MediaStreamTrack objects.

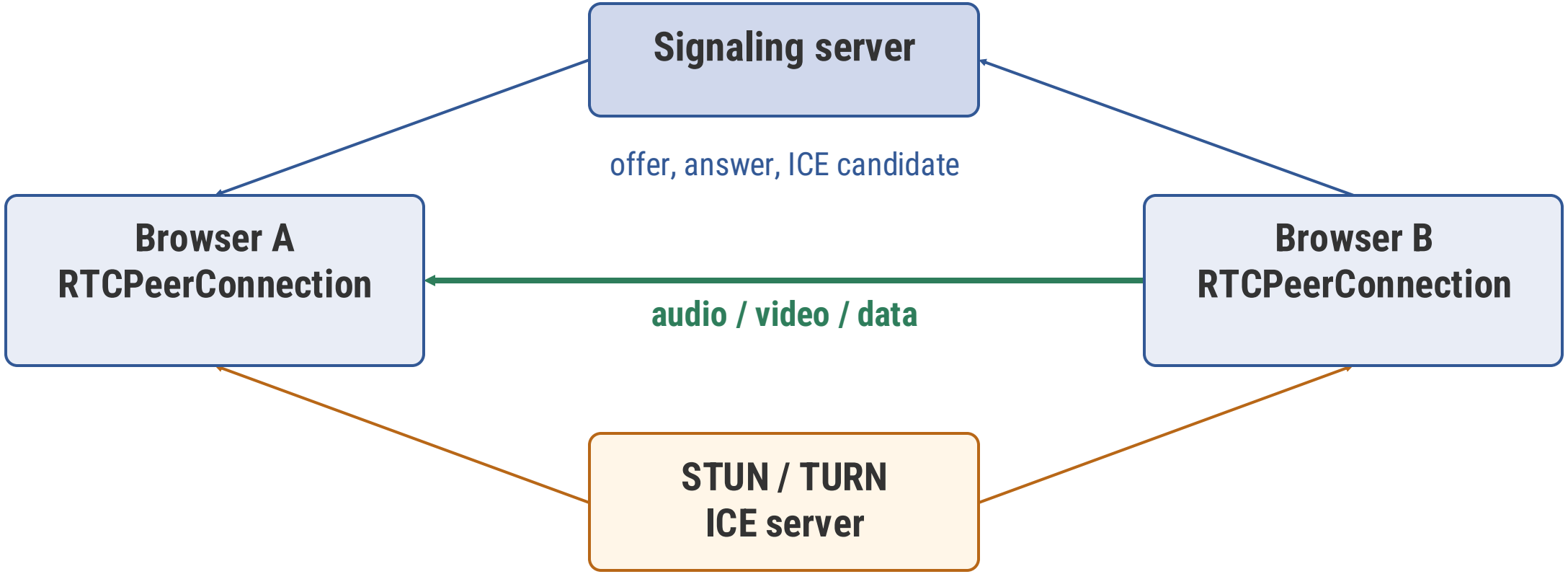
Peer connection

RTCPeerConnection manages negotiation, transport, and incoming tracks from a remote peer.

Data channel

RTCDataChannel carries application data such as chat messages, game state, or small files.

Basic WebRTC architecture



Signaling is done out-of-band (not specified in WebRTC standard).
You can use SDP, HTTP, Websocket, REST API, or even WhatsApp signaling.

Why?

Accessing the camera and microphone

First step

- Request permission to use the camera and microphone.
- The browser returns a MediaStream.
- Show the local stream before creating the connection.

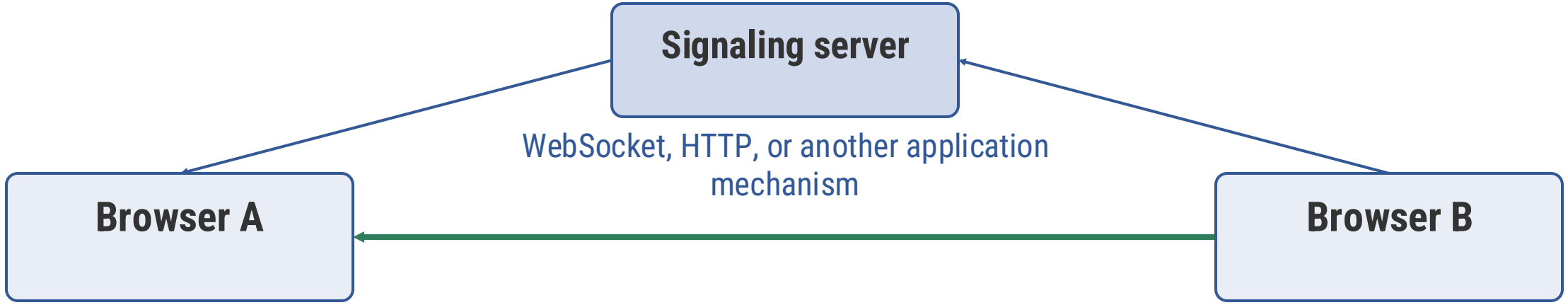
```
const stream = await
navigator.mediaDevices
  .getUserMedia({
    video: true,
    audio: true
  });

localVideo.srcObject = stream;
```

Camera and microphone streams contain sensitive data. The browser asks for consent and displays an indicator while a device is in use.

The browser must show an indicator (typically a green/red dot) that it uses the microphone/camera. This is part of the W3C security standard.

Signaling in a WebRTC application

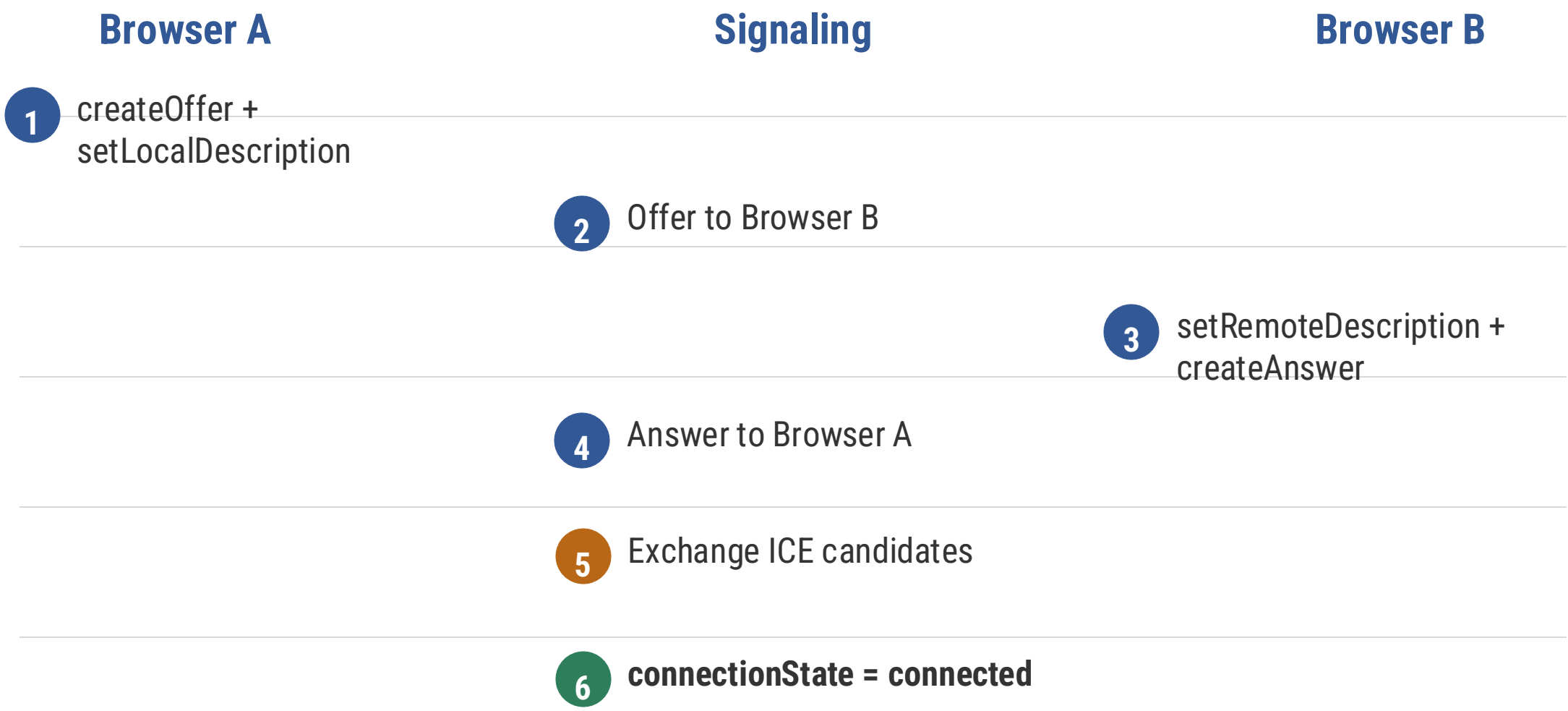


Signaling Exchanges SDP (Session Description Protocol) offers, answers, and ICE (Interactive Connectivity Establishment) candidates. WebRTC does not define the signaling transport.

Media Audio and video tracks flow after both peers agree on session parameters and a connection path.

Note: SDP is not part of WebRTC standard, but IETF standard back in 1998. However, SDP is commonly used in WebRTC applications.

Offer, answer, and ICE flow



Signaling, STUN, and TURN

Component	Primary function	Carries media?
Signaling Server	Exchanges offers, answers, and ICE candidates	No
STUN Server (Session Traversal Utilities for NAT)	Discovers a peer's public IP address and port when hidden behind NAT or home routers.	No
TURN Sever (Traversal Using Relays around NAT)	Acts as a relay server when direct P2P connection fails due to strict corporate firewalls.	Yes, as a relay

ICE gathers and tests candidates, then selects a working pair of connection paths.

Question: why do we need server if WebRTC is peer-to-peer?

Minimum APIs for a video call

```
const stream = await navigator.mediaDevices
  .getUserMedia({ video: true, audio: true });

const pc = new RTCPeerConnection({ iceServers });

stream.getTracks().forEach(track =>
  pc.addTrack(track, stream)
);

pc.ontrack = event => {
  remoteVideo.srcObject = event.streams[0];
};

pc.onicecandidate = event => {
  signaling.send(event.candidate);
};
```

What happens

- Capture local media.
- Add each track to the peer connection.
- Handle incoming remote tracks.
- Send ICE candidates through signaling.

The signaling code remains an abstraction in this example.

Production considerations

Permissions and privacy	Handle denied permissions. Clearly indicate when the camera or microphone is active.
Connectivity	Provide an authenticated TURN service. Test office, campus, cellular, and restrictive firewall networks.
Operations	Monitor connectionState, ICE state, bitrate, packet loss, and setup failures.
Multiparty calls	For larger calls, evaluate a media server such as an SFU so each browser does not send media to every peer.

Live demo: connecting two browsers

Demo sequence

- Open the application in two tabs or browsers.
- Allow camera and microphone access.
- Create a room and connect the second peer.
- Check the local and remote video.

What to observe

- Offer and answer messages in the log.
- ICE candidates arriving over time.
- connectionState changing to connected.
- Stopping a track turns off the camera.

Fallback for network problems: show the local camera preview, then continue with the offer and answer diagram.

Summary and knowledge check

1. **Does the signaling server carry the user's video?**

2. **How do STUN and TURN differ?**

3. **What do peers exchange before media starts flowing?**

WebRTC manages real-time communication. Signaling introduces the peers, ICE finds a path, and TURN relays traffic when needed.